



# Конкурсное задание

Республиканского конкурса профессионального  
мастерства WorldSkills Kazakhstan 2025

по компетенции Веб технологии

Module C: REST API

## Разработали:

Независимый эксперт

Афанасьева Дина

Главный эксперт

Попов Денис

Международный эксперт

Зияданова Айжан

## TABLE OF CONTENTS

|                               |   |
|-------------------------------|---|
| INTRODUCTION                  | 3 |
| PROJECT DESCRIPTION AND TASKS | 3 |
| MARKING SCHEME                | 6 |

## INTRODUCTION

Time to complete: 3 hours

You need to develop a REST API for a classifieds service — a geek marketplace. This is a platform where users can post and browse ads for selling LEGO sets, board games, books, souvenirs, etc.

In this module, you must implement role-based access: advert author, guest, moderator.

## PROJECT DESCRIPTION AND TASKS

### Competitor Information

A database dump is not provided; you must create the structure and populate the tables yourself.

All endpoints and validation rules are described in the file (media-files/swagger/openapi.yaml).

Create the following accounts and include them in the dump:

- Ethan Brooks, +1 202 555 0143, [ethan@ws-s17.kz](mailto:ethan@ws-s17.kz), <hash(ethan\_123)>, user
- Olivia Carter, +44 7700 900321, [olivia@ws-s17.kz](mailto:olivia@ws-s17.kz), <hash(olivia\_123)>, moderator

### Database

In the database directory of the project, store the database dump (DB\_DUMP.sql) and the ER diagram (ERD.png).

In the database dump, add test data for all tables so that the auto-tests pass successfully.

### General Rules for the API

- The order of properties in objects does not matter.
- The Content-Type response header must always be application/json, unless otherwise specified.
- Tokens for protected endpoints must be sent as **Bearer** tokens in the **Authorization** header.
- Timestamps must be formatted as ISO-8601 strings.  
For example: 2032-01-31T21:59:35.000Z. In some places, the required format is explicitly specified.

### Categories

| Field | Type and Constraints            |
|-------|---------------------------------|
| id    | Primary Key                     |
| name  | Unique name, max 100 characters |

## Users

| Field      | Type and Constraints                     |
|------------|------------------------------------------|
| id         | Primary Key                              |
| name       | Optional name, max 100 characters        |
| phone      | Unique phone number                      |
| email      | Unique email address                     |
| password   | Password (hash)                          |
| role       | Enum: user, moderatorDefault value: user |
| created_at | Creation timestamp                       |
| updated_at | Update timestamp                         |

## Adverts

| Field       | Type and Constraints                                                 |
|-------------|----------------------------------------------------------------------|
| id          | Primary Key                                                          |
| status      | Enum: draft, moderation, declined, published, archivedDefault: draft |
| title       | String, max 200 characters                                           |
| text        | String, max 1000 characters                                          |
| price       | Price                                                                |
| category_id | Foreign key referencing category                                     |
| user_id     | Foreign key referencing user                                         |
| photos      | JSON array of uploaded photo URLs                                    |
| created_at  | Creation timestamp                                                   |
| updated_at  | Update timestamp                                                     |

## Views

| Field     | Description   |
|-----------|---------------|
| advert_id | Advertisement |

| Field      | Description |
|------------|-------------|
| user_id    | User        |
| created_at |             |
| updated_at |             |

## REST API Requirements

### Auth

- During registration, a user specifies a phone number and an email.
- Login is possible using either phone number or email.
- Passwords must be stored in hashed form.
- Authorization uses a token consisting of at least 32 characters.
- A user can retrieve and update their profile.
- A user can retrieve a list of their adverts filtered by status.

### Categories

- Retrieving the list of categories does not require authorization.

### Adverts

- Retrieving a list of adverts and advert details does not require authorization.
- The adverts list request must support filters by category, price range, and text search. The text search must match advert titles, advert text, category name, and author's name.
- Adverts can be sorted by date or price in ascending or descending order.
- Sorting must take into account the VIP service, described later.
- When retrieving advert details by an authorized user, a record must be created in the views table. On subsequent views, a new record must not be created — instead, updated\_at must be updated.
- An authorized user can create an advert — it is created with the draft status. It must contain at least one photo.
- A user may edit their own adverts only if they are in the draft status.
- A moderator may edit any advert.
- Only the author can delete an advert, and only if it is in draft or archived status.

Status transitions:

- User:
  - draft → moderation
  - published → draft
  - declined → draft
  - published → archived
  - draft → archived
  - declined → archived

- Moderator:
  - moderation → published
  - moderation → declined
  - published → declined

## Advantage Services

In the standard search results, adverts are filtered by category and text and sorted by date/price. However, users should be able to promote their adverts higher in search results. For this, they can enable a VIP or TOP service.

The VIP service can be activated for one advert for 7 days and provides the following features:

- Regardless of sorting, the first 3 positions in the results must be random VIP adverts. They must not appear again later in the list.
- If a text filter is applied, VIP adverts that do **not** match the text filter must still appear in the results.
- If a price filter is applied, VIP adverts whose price falls outside the selected range by less than 100 must still appear. For example, if the filter is “up to 1200,” a VIP advert priced at 1290 must still appear.

The TOP service can be activated for one advert for 3 days and provides the following features:

- Regardless of sorting, the first position in the results is reserved for the TOP advert (before the VIP block). It must not appear again later in the list.
- Unlike VIP, TOP adverts are subject to all filters. If more than one TOP advert matches the criteria, they appear first and are sorted among themselves by the service activation time — the most recently activated ones first.

Your task is to create a table to store information about the connected services. The structure is up to you, but normalization will be evaluated.

You must also design endpoints for enabling, extending, and retrieving services, and update the **openapi.yaml** file accordingly. Save it in the swagger directory of the project.

Create a README.md file so we can understand your project and the new endpoints.

## MARKING SCHEME

|    | Description        | Assessment   |
|----|--------------------|--------------|
| C1 | Deliverables       | 1.50         |
| C2 | Authentication     | 3.00         |
| C3 | User               | 3.50         |
| C4 | Categories         | 1.00         |
| C5 | Adverts            | 9.50         |
| C6 | Advantage Services | 4.00         |
| C7 | Code quality       | 0.50         |
|    | <b>Total</b>       | <b>23.00</b> |